

NumPy

Dr. Sanjit Kumar Saha



Gisma
University
of Applied
Sciences



Data Arrays

[VanderPlas 2016]

<https://towardsdatascience.com/numpy-array-cookbook-generating-and-manipulating-arrays-in-python-2195c3988b09>

- Datasets are heterogenous
 - They can come from a wide range of sources and a wide range of formats
- However, it will help us to think of all data fundamentally as arrays of numbers
 - Images can be thought of as simply two-dimensional arrays of numbers representing pixel brightness across the area
 - Sound clips can be thought of as one-dimensional arrays of intensity versus time
 - Text can be converted in various ways into numerical representations
 - Efficient storage and manipulation of numerical arrays is absolutely fundamental
 - No matter what the data are, the first step in making them analyzable will be to transform them into arrays of numbers

1D Array

3	2
---	---

2D Array

1	0	1
3	4	1

3D Array

1	7	9
5	9	3
7	9	9



NumPy

- NumPy (short for Numerical Python) is package
 - That provides an efficient interface to store and operate on dense data buffers
 - NumPy arrays form the core of nearly the entire ecosystem of data science tools in Python
- NumPy arrays are like Python's built-in list type
 - But NumPy arrays provide much more efficient storage and data operations as the arrays grow larger in size
- `import numpy as np`
 - By convention, you will find that most people in the SciPy/PyData world will import NumPy using `np` as an alias

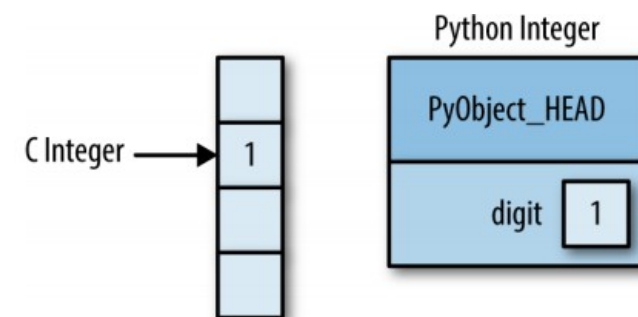


A Python Integer Is More Than Just an Integer

[VanderPlas 2016]

4

- The standard Python implementation is written in C
 - Every Python object is simply a cleverly disguised C structure, which contains not only its value, but other information as well
- A single integer in Python 3.4 contains four pieces
 - `ob_refcnt`, a reference count that helps Python silently handle memory allocation and deallocation
 - `ob_type`, which encodes the type of the variable
 - `ob_size`, which specifies the size of the following data members
 - `ob_digit`, which contains the actual integer value that we expect the Python variable to represent
- There is some overhead in storing an integer in Python
 - As compared to an integer in a compiled language like C



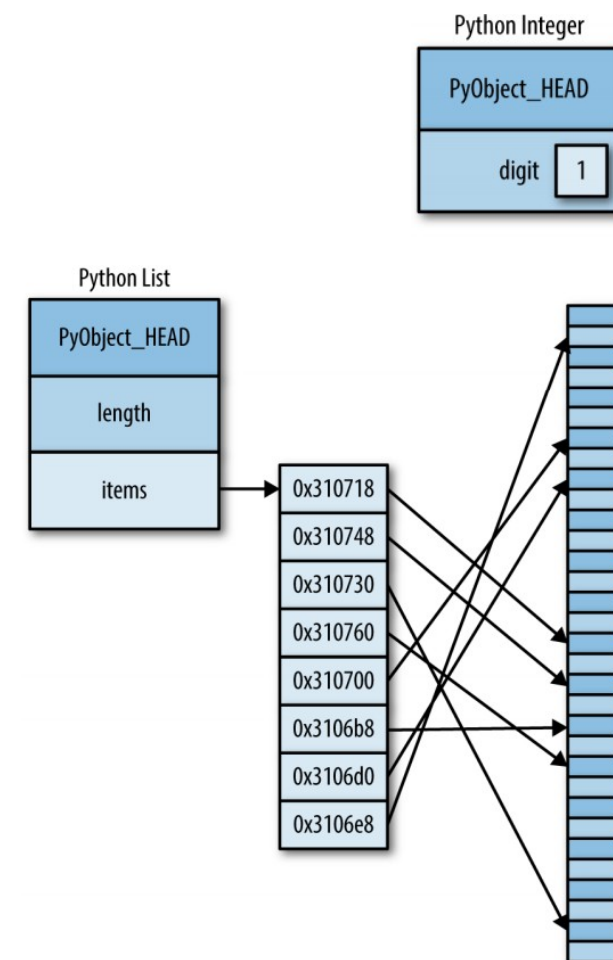


A Python List Is More Than Just a List

[VanderPlas 2016]

5

- The standard mutable multielement container in Python is the list
 - Because of Python's dynamic typing, we can even create heterogeneous lists
 - `my_list = [True, "2", 3.0, 4]`
- The Python list contains a pointer to a block of pointers
 - Each of which in turn points to a full Python object like the Python integer we saw earlier
 - Again, the advantage of the list is flexibility
 - Because each list element is a full structure containing both data and type information, the list can be filled with data of any desired type
- But this flexibility comes at a cost
 - To allow these flexible types, each item in the list must contain its own type info, reference count, and other information
 - That is, each item is a complete Python object
 - In the special case that all variables are of the same type, much of this information is redundant



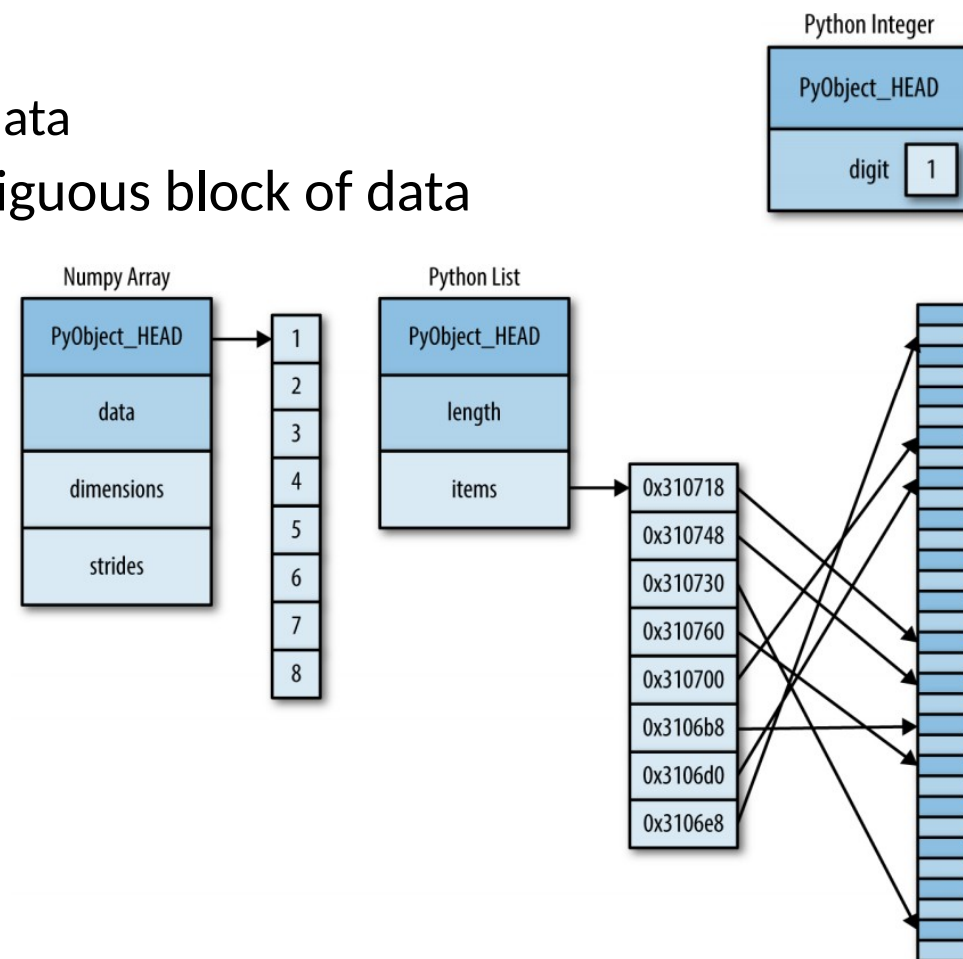


The Difference Between C and Python Lists

[VanderPlas 2016]

6

- It can be much more efficient to store data in a fixed-type array
 - Fixed-type NumPy-style arrays lack this flexibility
 - But are much more efficient for storing and manipulating data
 - The array essentially contains a single pointer to one contiguous block of data
 - At the implementation level





Fixed-Type Arrays in Python

- Python offers several different options for storing data in efficient, fixed-type data buffers
 - The built-in array module (available since Python 3.3) can be used to create dense arrays of a uniform type
 - Here “i” is a type code indicating the contents are integers
- While Python’s array object provides efficient storage of array-based data, NumPy adds to this efficient operations on that data

```
import array
L = list(range(10))
A = array.array("i",
L)
```



Creating Arrays from Python Lists

- `np.array([1, 4, 2, 5, 3])`
 - We can use `np.array` to create arrays from Python lists
 - Remember that unlike Python lists, NumPy is constrained to arrays that all contain the same type
 - If types do not match, NumPy will upcast if possible
 - E.g., integers are upcast to floating point
- `np.array([[1, 2, 3], [4, 5, 6]])`
 - Unlike Python lists, NumPy arrays can explicitly be multidimensional
 - The inner lists are treated as rows of the resulting two-dimensional array



NumPy Standard Data Types

- NumPy arrays contain values of a single type
 - So, it is important to have detailed knowledge of those types and their limitations
 - Because NumPy is built in C, the types will be familiar to users of C, Fortran, and other related languages
- We can specify them using a string or the associated NumPy object
 - `np.zeros(10, dtype="int16")`
 - `np.zeros(10, dtype=np.int16)`

Data type	Description
<code>bool_</code>	Boolean (True or False) stored as a byte
<code>int_</code>	Default integer type (same as C long; normally either <code>int64</code> or <code>int32</code>)
<code>intc</code>	Identical to C <code>int</code> (normally <code>int32</code> or <code>int64</code>)
<code>intp</code>	Integer used for indexing (same as C <code>ssize_t</code> ; normally either <code>int32</code> or <code>int64</code>)
<code>int8</code>	Byte (−128 to 127)
<code>int16</code>	Integer (−32768 to 32767)
<code>int32</code>	Integer (−2147483648 to 2147483647)
<code>int64</code>	Integer (−9223372036854775808 to 9223372036854775807)
<code>uint8</code>	Unsigned integer (0 to 255)
<code>uint16</code>	Unsigned integer (0 to 65535)
<code>uint32</code>	Unsigned integer (0 to 4294967295)
<code>uint64</code>	Unsigned integer (0 to 18446744073709551615)
<code>float_</code>	Shorthand for <code>float64</code>
<code>float16</code>	Half-precision float: sign bit, 5 bits exponent, 10 bits mantissa
<code>float32</code>	Single-precision float: sign bit, 8 bits exponent, 23 bits mantissa
<code>float64</code>	Double-precision float: sign bit, 11 bits exponent, 52 bits mantissa
<code>complex_</code>	Shorthand for <code>complex128</code>
<code>complex64</code>	Complex number, represented by two 32-bit floats
<code>complex128</code>	Complex number, represented by two 64-bit floats



Creating Arrays from Scratch

- Especially for larger arrays, it is more efficient to create arrays from scratch using routines built into NumPy
 - `np.zeros(10, dtype=int)`
 - `np.ones((3, 5), dtype=float)`
 - `np.full((3, 5), 3.14)`
 - `np.arange(0, 20, 2)`
 - `np.linspace(0, 1, 5)`
 - `np.random.random((3, 3))`
 - `np.random.normal(0, 1, (3, 3))`
 - `np.random.randint(0, 10, (3, 3))`
 - `np.eye(3)`
 - `np.empty(3)`



Array Attributes

- Each array has attributes
 - *ndim*
 - The number of dimensions
 - *shape*
 - The size of each dimension
 - *size*
 - The total size of the array
 - *dtype*
 - The data type of the array
 - *itemsize*
 - The size (in bytes) of each array element
 - *nbytes*
 - The total size (in bytes) of the array

```
np.random.seed(0)
x3 = np.random.randint(10, size=(3, 4, 5))
print("x3 ndim: ", x3.ndim)
print("x3 shape:", x3.shape)
print("x3 size: ", x3.size)
print("dtype:", x3.dtype)
print("itemsize:", x3.itemsize, "bytes")
print("nbytes:", x3.nbytes, "bytes")
```



Array Indexing: Accessing Single Elements

- Indexing in NumPy will feel quite familiar to Python's standard list indexing
 - `x1 = np.random.randint(10, size=6)`
 - `x2 = np.random.randint(10, size=(3, 4))`
 - You can also modify values using any of the below index notation
- `x1[4]`
 - In a one-dimensional array, you can access the *i*th value
- `x1[-2]`
 - To index from the end of the array, you can use negative indices
- `x2[2, 0]`
 - In a multidimensional array, you access items using a comma-separated tuple of indices



Array Slicing: Accessing Subarrays

- `x[start:stop:step]`
 - We can also access subarrays with the slice notation, marked by the colon (:) character
 - The NumPy slicing syntax follows that of the standard Python list
 - They default to the values start=0, stop=size of dimension, step=1
- Examples for 1D arrays
 - `x[:5]` *# first five elements*
 - `x[::2]` *# every other element*
 - `x[::-1]` *# all elements, reversed*
 - `x[5::-2]` *# reversed every other from index 5*
- Examples for multidimensional arrays
 - `x2[:2, :3]` *# two rows, three columns*
 - `x2[:3, ::2]` *# all rows, every other column*
 - `x2[::-1, ::-1]`



Array Slicing: Accessing Subarrays

- We can also access single rows or columns of an array by combining indexing and slicing
 - Using an empty slice marked by a single colon (:)
- Examples
 - `x2[:, 0]` *# first column of x2*
 - `x2[0, :]` *# first row of x2*
 - `x2[0]` *# equivalent to x2[0, :]*



Subarrays as No-Copy Views

- Array slices return views rather than copies of the array data
 - This is one area in which NumPy array slicing differs from Python list slicing as in lists, slices will be copies
 - If we modify a subarray, we will see that the original array is changed
 - `x2_sub = x2[:2, :2]`
 - `x2_sub[0, 0] = 99`
 - `print(x2)`
 - This default behavior is actually quite useful
 - When we work with large datasets, we can access and process pieces of these datasets without the need to copy the underlying data buffer
- `x2_sub_copy = x2[:2, :2].copy()`
 - However, we can instead explicitly copy the data within an array or a subarray by the `copy()` method
 - If we now modify this subarray, the original array is not touched



Reshaping of Arrays

- `grid = np.arange(1, 10).reshape((3, 3))`
 - Another useful type of operation is reshaping of arrays using the `reshape()` method
 - The above code puts the numbers 1 through 9 in a 3×3 grid
 - Note that the size of the initial array must match the size of the reshaped array
- Another common pattern is the conversion of a 1D array into a 2D row/column matrix
 - Using the `reshape` method or the `newaxis` keyword within a slice operation
 - `x = np.array([1, 2, 3])`
 - `x.reshape((1, 3))` *# row vector via reshape*
 - `x[np.newaxis, :]` *# row vector via newaxis*
 - `x.reshape((3, 1))` *# column vector via reshape*
 - `x[:, np.newaxis]` *# column vector via newaxis*



Array Concatenation

- *np.concatenate* takes a tuple or list of arrays as its first argument
 - *x = np.array([1, 2, 3])*
 - *y = np.array([3, 2, 1])*
 - *np.concatenate([x, y])*
- *np.concatenate* can also be used for two-dimensional arrays
 - *grid = np.array([[1, 2, 3], [4, 5, 6]])*
 - *np.concatenate([grid, grid])* *# concatenate along the first axis*
 - *np.concatenate([grid, grid], axis=1)* *# concatenate along the second axis*



Array Concatenation

- It can be clearer to use vertical/horizontal stack methods
 - Similarly, `np.dstack` will stack arrays along the third axis
- `np.vstack`
 - `x = np.array([1, 2, 3])`
 - `grid = np.array([[9, 8, 7], [6, 5, 4]])`
 - `np.vstack([x, grid])` *# vertically stack the arrays*
- `np.hstack` (horizontal stack)
 - `y = np.array([[99], [99]])`
 - `np.hstack([grid, y])` *# horizontally stack the arrays*



Array Splitting

- The opposite of concatenation is splitting, which is implemented by the functions *np.split*, *np.hsplit*, and *np.vsplit*
 - $x = [1, 2, 3, 99, 99, 3, 2, 1]$
 - $x1, x2, x3 = np.split(x, [3, 5])$
- For each of these, we can pass a list of indices giving the split points
 - Notice that N split points lead to $N + 1$ subarrays
- The related functions *np.hsplit* and *np.vsplit* are similar
 - $grid = np.arange(16).reshape((4, 4))$
 - $upper, lower = np.vsplit(grid, [2])$
 - $left, right = np.hsplit(grid, [2])$
- Similarly, *np.dsplit* will split arrays along the third axis



Computation on NumPy Arrays: Universal Functions

- Computation on NumPy arrays can be very fast, or it can be very slow
 - It depends on how we use it
- The key to making it fast is to use vectorized operations
 - They are generally implemented through NumPy's universal functions (ufuncs)
 - It can be used to make repeated calculations on array elements much more efficient



The Slowness of Loops

- Python's default implementation (known as CPython) does some operations very slowly
 - This is in part due to the dynamic, interpreted nature of the language
 - The fact that types are flexible, so that sequences of operations cannot be compiled down to efficient machine code as in languages like C and Fortran
 - The relative sluggishness of Python generally manifests itself in situations where many small operations are being repeated
 - For instance, looping over arrays to operate on each element
- This implementation probably feels fairly natural to someone from, say, a C or Java background
 - But this operation is very slow

```
output = np.empty(len(values))  
for i in range(len(values)):  
    output[i] = values[i] / 2.0
```





UFuncs

- Vectorized operations
 - NumPy provides a convenient interface into just this kind of statically typed, compiled routine
 - This is known as a vectorized operation
 - This vectorized approach is designed to push the loop into the compiled layer that underlies NumPy
 - Leading to much faster execution
 - Vectorized operations in NumPy are implemented via ufuncs
 - Whose main purpose is to quickly execute repeated operations on values in NumPy arrays
- *output = values / 2.0*
 - We can accomplish the same operation by simply performing an operation on the array, which will then be applied to each element
- Ufuncs are extremely flexible
 - We can also operate between two arrays
 - `np.arange(5) / np.arange(1, 6)`
 - Ufunc operations are not limited to one-dimensional arrays
 - `x = np.arange(9).reshape((3, 3))`
 - `2 ** x`

```
output = np.empty(len(values))  
for i in range(len(values)):  
    output[i] = values[i] / 2.0
```



Array Arithmetic

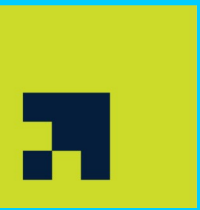
- NumPy's ufuncs feel very natural to use
 - Because they make use of Python's native arithmetic operators
 - `x = np.arange(4)`
 - `x + 5`
 - `-x`
 - `x ** 2`
- All of these arithmetic operations are simply convenient wrappers
 - Around specific functions built into NumPy
 - For example, the `+` operator is a wrapper for the `add` function
 - `np.add(x, 2)`

Operator	Equivalent ufunc	Description
+	<code>np.add</code>	Addition (e.g., <code>1 + 1 = 2</code>)
-	<code>np.subtract</code>	Subtraction (e.g., <code>3 - 2 = 1</code>)
-	<code>np.negative</code>	Unary negation (e.g., <code>-2</code>)
*	<code>np.multiply</code>	Multiplication (e.g., <code>2 * 3 = 6</code>)
/	<code>np.divide</code>	Division (e.g., <code>3 / 2 = 1.5</code>)
//	<code>np.floor_divide</code>	Floor division (e.g., <code>3 // 2 = 1</code>)
**	<code>np.power</code>	Exponentiation (e.g., <code>2 ** 3 = 8</code>)
%	<code>np.mod</code>	Modulus/remainder (e.g., <code>9 % 4 = 1</code>)



Absolute Value

- NumPy also understands Python's built-in absolute value function
 - `x = np.array([-2, -1, 0, 1, 2])`
 - `abs(x)`
- The corresponding NumPy *ufunc* is `np.absolute`
 - Which is also available under the alias `np.abs`
 - `np.absolute(x)`
 - `np.abs(x)`



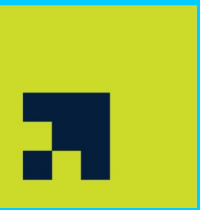
Trigonometric Functions

- NumPy provides a large number of useful ufuncs, including trigonometric functions
 - `theta = np.linspace(0, np.pi, 3)` *# defining an array of angles*
 - `print("theta = ", theta)`
 - `print("sin(theta) = ", np.sin(theta))`
 - `print("cos(theta) = ", np.cos(theta))`
 - `print("tan(theta) = ", np.tan(theta))`
- Inverse trigonometric functions are also available
 - `x = [-1, 0, 1]`
 - `print("x = ", x)`
 - `print("arcsin(x) = ", np.arcsin(x))`
 - `print("arccos(x) = ", np.arccos(x))`
 - `print("arctan(x) = ", np.arctan(x))`



Exponents and Logarithms

- Another common type of operation available in a NumPy ufunc are the exponentials
 - `x = [1, 2, 3]`
 - `print("x =", x)`
 - `print("e^x =", np.exp(x))`
 - `print("2^x =", np.exp2(x))`
 - `print("3^x =", np.power(3, x))`
- The inverse of the exponentials, the logarithms, are also available
 - `x = [1, 2, 4, 10]`
 - `print("x =", x)`
 - `print("ln(x) =", np.log(x))`
 - `print("log2(x) =", np.log2(x))`
 - `print("log10(x) =", np.log10(x))`



Aggregations: Min, Max, and Everything in Between

- NumPy has fast built-in aggregation functions for working on arrays
 - `L = np.random.random(100)`
 - `np.sum(L)`
 - A shorter syntax is to use methods of the array object itself
 - `L.sum()`
- Be careful that the `sum` function and the `np.sum` function are not identical
 - Which can sometimes lead to confusion
 - Whenever possible, make sure that you are using the NumPy version of these aggregates when operating on NumPy arrays
 - NumPy's corresponding functions have similar syntax, and again operate much more quickly than their similar built-in functions

Function Name	NaN-safe Version	Description
<code>np.sum</code>	<code>np.nansum</code>	Compute sum of elements
<code>np.prod</code>	<code>np.nanprod</code>	Compute product of elements
<code>np.mean</code>	<code>np.nanmean</code>	Compute median of elements
<code>np.std</code>	<code>np.nanstd</code>	Compute standard deviation
<code>np.var</code>	<code>np.nanvar</code>	Compute variance
<code>np.min</code>	<code>np.nanmin</code>	Find minimum value
<code>np.max</code>	<code>np.nanmax</code>	Find maximum value
<code>np.argmin</code>	<code>np.nanargmin</code>	Find index of minimum value
<code>np.argmax</code>	<code>np.nanargmax</code>	Find index of maximum value
<code>np.median</code>	<code>np.nanmedian</code>	Compute median of elements
<code>np.percentile</code>	<code>np.nanpercentile</code>	Compute rank-based statistics of elements
<code>np.any</code>	N/A	Evaluate whether any elements are true
<code>np.all</code>	N/A	Evaluate whether all elements are true



Multidimensional Aggregates

- By default, each NumPy aggregation function will return the aggregate over the entire array
 - `M = np.random.random((3, 4))`
 - `M.sum()`
- Aggregation functions take an additional argument specifying the axis
 - Along which the aggregate is computed
 - `M.min(axis=0)` *# minimum value within each column*
 - `M.max(axis=1)` *# maximum value within each row*
 - The `axis` keyword specifies the dimension of the array that will be collapsed, rather than the dimension that will be returned



Computation on Arrays: Broadcasting

[VanderPlas 2016]

29

- Broadcasting is simply a set of rules for applying binary UFuncs on arrays of different sizes
 - For example, `np.arange(3) + 5`
 - We can think of this as an operation that stretches or duplicates the value 5 into the *array* `[5, 5, 5]`, and adds the results
 - We can similarly extend this to arrays of higher dimension
- The light boxes represent the broadcasted values
 - This extra memory is not actually allocated in the course of the operation, but it can be useful conceptually to imagine that it is

`np.arange(3)+5`

0	1	2
---	---	---

 +

5	5	5
---	---	---

 =

5	6	7
---	---	---

`np.ones((3, 3))+np.arange(3)`

1	1	1
1	1	1
1	1	1

 +

0	1	2
0	1	2
0	1	2

 =

1	2	3
1	2	3
1	2	3

`np.ones((3, 1))+np.arange(3)`

0
1
2

 +

0	1	2
0	1	2
0	1	2

 =

0	1	2
1	2	3
2	3	4



Rules of Broadcasting

[VanderPlas 2016]

30

- Rule 1

- If the two arrays differ in their number of dimensions, the shape of the one with fewer dimensions is padded with ones on its leading (left) side
 - $M.shape: (2, 3) \rightarrow (2, 3)$
 - $a.shape: (3,) \rightarrow (1, 3)$

$np.arange(3)+5$

0	1	2
---	---	---

 +

5	5	5
---	---	---

 =

5	6	7
---	---	---

- Rule 2

- If the shape of the two arrays does not match in any dimension, the array with shape equal to 1 in that dimension is stretched to match the other shape
 - $M.shape: (2, 3) \rightarrow (2, 3)$
 - $a.shape: (1, 3) \rightarrow (2, 3)$

$np.ones((3, 3))+np.arange(3)$

1	1	1
1	1	1
1	1	1

 +

0	1	2
0	1	2
0	1	2

 =

1	2	3
1	2	3
1	2	3

- Rule 3

- If in any dimension the sizes disagree and neither is equal to 1, an error is raised
 - $M.shape: (3, 2)$
 - $a.shape: (3, 3)$

$np.ones((3, 1))+np.arange(3)$

0	0	0
1	1	1
2	2	2

 +

0	1	2
0	1	2
0	1	2

 =

0	1	2
1	2	3
2	3	4



Comparisons, Masks, and Boolean Logic

- Masking comes up when you want to extract, modify, count, or otherwise manipulate values in an array based on some criterion
 - Counting all values greater than a certain value
 - Removing all outliers that are above some threshold
- In NumPy, Boolean masking is often the most efficient way to accomplish these types of tasks



Comparison Operators as UFuncs

- NumPy also implements comparison operators as element-wise ufuncs
 - `x = np.array([1, 2, 3, 4, 5])`
 - `x < 3` *# less than*
 - The result of these comparison operators is always an array with a Boolean data type
 - `[True, True, False, False, False]`
- It is also possible to do an element-by-element comparison of two arrays
 - And to include compound expressions
 - These will work on arrays of any size and shape
 - `(2 * x) == (x ** 2)`

Operator	Equivalent ufunc
<code>==</code>	<code>np.equal</code>
<code>!=</code>	<code>np.not_equal</code>
<code><</code>	<code>np.less</code>
<code><=</code>	<code>np.less_equal</code>
<code>></code>	<code>np.greater</code>
<code>>=</code>	<code>np.greater_equal</code>



Working with Boolean Arrays

- We can count the number of True entries in a Boolean array
 - `np.count_nonzero(x < 6)` # how many values less than 6?
 - Another way to get at this information is to use `np.sum`
 - *False* is interpreted as 0, and *True* is interpreted as 1
 - `np.sum(x < 6)`
 - `np.sum(x < 6, axis=1)` # how many values less than 6 in each row?
- We can check whether any or all the values are true using `np.any()` or `np.all()`
 - `np.any(x > 8)` # are there any values greater than 8?
 - `np.all(x < 10)` # are all values less than 10?
 - `np.all(x < 8, axis=1)` # are all values in each row less than 8?



Bitwise Logic Operators

- NumPy overloads bitwise logic operators as ufuncs that work element-wise on (usually Boolean) arrays
 - `inches = np.array([0.4, 0.7, 0.9, 1.3, 1.5])`
 - `np.sum((inches > 0.5) & (inches < 1))`
 - Note that the parentheses here are important
 - Because of operator precedence rules
- Remember that
 - `and` and `or` perform a single Boolean evaluation on an entire object
 - `&` and `|` perform multiple Boolean evaluations on the content (the individual bits or bytes) of an object
 - For Boolean NumPy arrays, the latter is nearly always the desired operation

Operator	Equivalent ufunc
<code>&</code>	<code>np.bitwise_and</code>
<code> </code>	<code>np.bitwise_or</code>
<code>^</code>	<code>np.bitwise_xor</code>
<code>~</code>	<code>np.bitwise_not</code>



Boolean Arrays as Masks

- $x < 5$
 - Suppose we want an array of all values in the array that are less than 5
 - We can obtain a Boolean array for this condition easily
- $x[x < 5]$
 - Now to select these values from the array, we can simply index on this Boolean array
- This is known as a masking operation
 - What is returned is a one-dimensional array filled with all the values that meet this condition
 - All the values in positions at which the mask array is *True*



Fancy Indexing

- Fancy indexing means passing an array of indices to access multiple array elements at once
 - The difference to normal indexing is that, here, we pass arrays of indices in place of single scalars
 - This allows us to very quickly access and modify complicated subsets of an array's values
 - With fancy indexing, the shape of the result reflects the shape of the index arrays
 - Rather than the shape of the array being indexed
- $x[3], x[7], x[2]$
 - Suppose we want to access three different elements
- $ind = [3, 7, 4]$
- $x[ind]$
 - Alternatively, we can pass a single list or array of indices to obtain the same result





Fancy Indexing

- Like with standard indexing, the first index refers to the row, and the second to the column
 - `X = np.arange(12).reshape((3, 4))`
 - `row = np.array([0, 1, 2])`
 - `col = np.array([2, 1, 3])`
 - `X[row, col]`
- It can also be used to modify parts of an array
 - `x = np.arange(10)`
 - `i = np.array([2, 1, 8, 4])`
 - `x[i] = 99`



Sorting Arrays

- NumPy's `np.sort` function turns out to be much more efficient than Python's built-in `sort`
 - `x = np.array([2, 1, 4, 3, 5])`
- `np.sort(x)`
 - To return a sorted version of the array without modifying the input, you can use `np.sort`
- `x.sort()`
 - If you prefer to sort the array in-place, you can instead use the `sort` method of arrays
- `i = np.argsort(x)`
 - A related function is `argsort`, which instead returns the indices of the sorted elements



Sorting Along Rows or Columns

- A useful feature of NumPy's sorting algorithms is the ability to sort along specific rows or columns
 - Of a multidimensional array using the axis argument
 - `np.sort(X, axis=0)` *# sort each column of X*
 - `np.sort(X, axis=1)` *# sort each row of X*
- Keep in mind that this treats each row or column as an independent array
 - And any relationships between the row or column values will be lost